BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC

HỌC PHẦN: MÃ HÓA VÀ ỨNG DỤNG

CHỦ ĐỀ 8: PHÂN TÍCH VÀ PHÁ MÃ VIGENÈRE BẰNG PHÂN TÍCH TẦN SUẤT

TP. Hồ Chí Minh, Tháng 11/2025

BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC

HỌC PHẦN: MÃ HÓA VÀ ỨNG DỤNG

CHỦ ĐỀ 8: PHÂN TÍCH VÀ PHÁ MÃ VIGENÈRE BẰNG PHÂN TÍCH TẦN SUẤT

GVHD: Hồ Hải Quân

Nhóm 5:

- Võ Phát Đạt – 2033230063 (nhóm trưởng)
- Đào Phan Quốc Anh – 2033230011
- Phạm Văn Quý – 2033230242

TP. Hồ Chí Minh, Tháng 11/2025

LỜI CAM ĐOAN

Nhóm trưởng nhóm 5 đại diện cho nhóm xin cam đoan tất cả các nội dung của báo cáo đồ án này đều được thực hiện bởi các thành viên của nhóm. Các số liệu, hình ảnh và kết quả phân tích trong báo cáo đều được thực hiện trung thực một cách rõ ràng và có để lại nguồn trích dẫn rõ ràng. Những nội dung tham khảo từ tài liệu, giáo trình, trang web hoặc các nguồn khác đều đã được ghi rõ trong phần tài liệu tham khảo. Nhóm sẽ chịu mọi hoàn toàn trách nhiệm về tính trung thực của báo cáo.

Đại diện nhóm

Võ Phát Đạt

MỤC LỤC

LỜI MỞ ĐẦU **6**
PHỤ LỤC HÌNH ẢNH **7**
PHỤ LỤC BẢNG **8**
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT MẬT MÃ VIGENÈRE **9**
1.1 Giới thiệu 9
1.2 Nguyên lý 9
1.3 Phân tích độ an toàn và hạn chế 10
CHƯƠNG 2. PHƯƠNG PHÁP KASISKI **11**
2.1 Giới thiệu 11
2.2 Phương pháp triển khai 11
2.3 Ưu điểm và hạn chế 12
CHƯƠNG 3. KỸ THUẬT PHÂN TÍCH TẦN SUẤT **14**
3.1 Giới thiệu 14
3.2 Cơ sở lý thuyết và triển khai 14
3.3 Kiểm định Chi – Bình phương (Chi-squared Test) 16
3.3.1 Cơ sở lý thuyết 16
3.3.2 Các bước triển khai 18
CHƯƠNG 4. THỰC HIỆN GIẢI MÃ CHI TIẾT **19**
4.1 Thực hiện giải mã thủ công 19
4.1.1 Ước lượng độ dài khóa với Kasiski 19
4.1.2 Thực hiện phân tích tần suất với bản mã (độ dài khóa là 3) 19
4.1.3 Tập hợp khóa và giải mã bản mã 28
4.2 Giải mã bằng chương trình 30
4.2.1 Giới thiệu và cấu trúc tổng thể của chương trình 30
4.2.2 Cấu trúc tổng thể của chương trình 30
4.2.3 Thực hiện giải mã với chương trình 41

4.3 Nhận xét và tổng kết 44
4.3.1 Ưu điểm của chương trình 44
4.3.2 Một số hạn chế của chương trình 45
4.3.3 Nhận xét chung 47
TÀI LIỆU THAM KHẢO 48

LỜI MỞ ĐẦU

Trong thế giới An toàn thông tin đang phát triển với tốc độ chóng mặt, nơi các hệ thống mật mã hiện đại luôn là tuyến phòng thủ cuối cùng, việc nghiên cứu về các hệ mật cổ điển không chỉ là một bài học lịch sử mà còn là nền tảng cốt lõi để đào sâu vào tư duy phản biện. Chúng ta không chỉ học cách xây dựng, mà còn phải học cách phá vỡ chúng để hiểu rõ điểm yếu và từ đó thiết kế nên những hệ thống bảo mật vững chắc hơn.

Mật mã Vigenère – còn được mệnh danh là Le Chiffre Indéchiffrable - "Mật mã không thể giải mã" – từng là một thách thức lớn trong lịch sử mật mã học nhờ cơ chế đa bảng phức tạp hơn hẳn so với Caesar đơn giản. Tuy nhiên, mọi hệ thống đều có điểm yếu và Vigenère cũng không ngoại lệ. Báo cáo này đi sâu vào việc khai thác chính điểm yếu đó.

Bài báo cáo này tập trung vào việc thực hành và kết hợp hai kỹ thuật phân tích mạnh mẽ: Phương pháp Kasiski để ước lượng độ dài khóa và Kỹ thuật Phân tích Tần suất được nâng cấp bằng Kiểm định Chi - bình phương để xác định khóa mã hóa một cách chính xác và khách quan.Thông qua việc giải mã một bản mã Vigenère cụ thể, báo cáo hy vọng sẽ minh chứng được sự giao thoa giữa Lý thuyết Mật mã và Thống kê Toán học.

Mục tiêu cuối cùng là trang bị cái nhìn sâu sắc, thực tế về cách mà những lỗ hổng mang tính chu kỳ (periodicity) và thống kê ngôn ngữ có thể được tận dụng để vượt qua rào cản tưởng chừng như bất khả xâm phạm của mật mã cổ điển.

PHỤ LỤC HÌNH ẢNH

Hình 3.1 Tần suất xuất hiện của các chữ cái trong một số ngôn ngữ tự nhiên 14
Hình 3.2 Tần suất các chữ cái xuất hiện (Aphalbet) giảm dần.....15
Hình 4.1 Thư mục chương trình PhaMaVigenege.....30
Hình 4.2 Bên trong thư mục module 31
Hình 4.3 Hàm find_repeated_sequences 32
Hình 4.4 Các hàm phục vụ cho việc tính ước chung.....32
Hình 4.5 Hàm kasiski_analysis 34
Hình 4.6 Hàm analyze_key_length phân tích độ dài khóa cho trước 36
Hình 4.7 Hàm chi_squared_test để tính Chi bình phương 36
Hình 4.8 Hàm decrypt_full.....37
Hình 4.9 Hàm decrypt_caesar 37
Hình 4.10 Mã hóa Vigenère.....38
Hình 4.11 Giải mã Vigenère khi biết được khóa 38
Hình 4.12 Giải mã Vigenère khi chưa biết khóa bằng phân tích tần suất 39
Hình 4.13 Hàm encrypt_vigenere() và decrypt_vigenere ().....40
Hình 4.14 Một danh sách gần 500 từ tiếng anh xuất hiện nhiều nhất trong văn bản.....41
Hình 4.15 Biến score dùng để tính toán điểm thông qua chi_total và cw_count.....41
Hình 4.16 Văn bản ngẫu nhiên 42
Hình 4.17 Kết quả sau khi chạy phân tích tần suất 43
Hình 4.18 Chi tiết kết quả phân tích tần suất 43
Hình 4.19 Mã hóa bản rõ với khóa RANDOMKEY 46
Hình 4.20 Trường hợp điển hình khiến cho chương trình không giải mã được.....46

PHỤ LỤC BẢNG

Bảng 1 Bảng tần suất ký tự trong Tiếng Anh 18
Bảng 2 Kết quả tính O_i ở K_1 21
Bảng 3 Kết quả sau khi tính O_i ở mọi lần dịch (K_1) 22
Bảng 4 Kết quả Chi bình phương ở K_1 23
Bảng 5 Kết quả sau khi tính O_i ở mọi lần dịch (K_2) 24
Bảng 6 Kết quả Chi bình phương ở K_2 25
Bảng 7 Kết quả sau khi tính O_i ở mọi lần dịch (K_3) 27
Bảng 8 Kết quả Chi bình phương ở K_3 28

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT MẬT MÃ VIGENÈRE

1.1 Giới thiệu

Mật mã Vigenère là một trong những phương pháp mã hóa cổ điển nổi tiếng, thuộc loại mật mã đa bảng (polyalphabetic cipher). Khác với mật mã đơn bảng như Caesar, nơi mỗi ký tự trong bản rõ được thay thế bằng một ký tự duy nhất theo một quy luật cố định, mật mã Vigenère sử dụng nhiều bảng thay thế để tăng độ phức tạp và khó bị giải mã bằng phương pháp phân tích tần suất thông thường.

1.2 Nguyên lý

Nguyên lý cơ bản của mật mã Vigenère dựa trên việc lặp lại một khóa chữ cái (key) có độ dài cố định để thay thế từng ký tự trong bản rõ. Cụ thể, mỗi ký tự thứ i trong bản rõ P_i được dịch chuyển theo vị trí tương ứng theo giá trị của ký tự thứ i trong khóa K_i. Khi khóa kết thúc, quá trình lặp lại từ đầu khóa sẽ tiếp tục cho đến hết bản rõ. Nhờ cơ chế này, cùng một ký tự xuất hiện nhiều lần trong bản rõ có thể được mã hóa thành nhiều ký tự khác nhau trong bản mã, làm giảm khả năng giải mã bằng phân tích tần suất truyền thống.

Giả sử Alphabet gồm các chữ cái Alphabet biểu diễn theo số (A → 0, B → 1, … , Z → 25), phép mã hóa sử dụng công thức như sau:

$$\boxed{C_i = (P_i + K_i) \bmod 26}$$

Trong đó:

- P_i là giá trị số của ký tự thứ i trong bản Alphabet của bản rõ P
- K_i là giá trị số của ký tự thứ i trong bản Alphabet của khóa K
- C_i là giá trị số của ký tự thứ i trong bản mã C sau khi tính tương ứng

Khi triển khai, chúng ta cần quy định rõ là các chỉ số của Alphabet bắt đầu từ 0 hoặc 1.

Ví dụ, ta có bản rõ là “HELLO” và khóa là “KEY” quá trình mã hóa sẽ được thực hiện như sau:

PlaintText	H	E	L	L	O
Key	K	E	Y	K	E
P	7	4	11	11	14
K	10	4	24	10	4
$C_i = (P_i + K_i)$ mode 26	17	8	9	21	18
Ciphertext	R	I	J	V	S

Bản mã của “HELLO” với khóa là “KEY” là “RIJVS”

1.3 Phân tích độ an toàn và hạn chế

Mật mã Vigenère từng được xem là “khó phá” so với Caesar nhờ cơ chế đa bảng. Tuy nhiên, nếu độ dài khóa ngắn hoặc khóa được tái sử dụng nhiều lần, Vigenère có thể bị phá bằng các phương pháp sau:

- Phương pháp Kasiski: tìm các đoạn lặp trong bản mã, đo khoảng cách giữa các đoạn giống nhau để ước lượng bội số của độ dài khóa.
- Chỉ số đồng nhất (Index of Coincidence - IC): so sánh tần suất xuất hiện chữ cái để ước tính xem một chuỗi thuộc về văn bản mã hóa bằng nhiều bảng hay chỉ một bảng.
- Phân tích tần suất theo từng “bảng”: nếu biết hoặc ước lượng được độ dài khóa m, ta có thể tách bản mã thành m chuỗi con tương ứng từng vị trí trong khóa và áp dụng phân tích tần suất (giống như phá Caesar) cho từng chuỗi con.

Tóm lại, Vigenère an toàn khi khóa đủ dài, ngẫu nhiên và không tái sử dụng (về lý thuyết nếu khóa bằng hoặc dài hơn bản rõ và hoàn toàn ngẫu nhiên thì nó trở thành One-Time Pad an toàn tuyệt đối). Nhưng trong thực tế thông thường (khóa ngắn/tái sử dụng), Vigenère có điểm yếu rõ rệt.

CHƯƠNG 2. PHƯƠNG PHÁP KASISKI

2.1 Giới thiệu

Phương pháp Kasiski (Kasiski Examination) là phương pháp phân tích dùng để ước lượng độ dài khóa của mật mã Vigenère khi khóa ngắn hơn bản rõ. Khi biết (hoặc thu hẹp) tập ứng viên độ dài khóa, ta có thể tách ciphertext theo vị trí modulo m rồi dùng phân tích tần suất để tìm khóa thực.

2.2 Phương pháp triển khai

Khi khóa của mật mã Vigenère ngắn hơn bản rõ, nó sẽ được lặp lại nhiều lần trên toàn bộ chuỗi cần mã hóa. Do đó, nếu trong bản rõ có hai đoạn giống nhau xuất hiện ở các vị trí cách nhau đúng bằng bội số của độ dài khóa, thì hai đoạn đó sẽ bị mã hóa bằng cùng một chuỗi khóa và cho ra cùng một đoạn trong bản mã.

Nói cách khác, nếu hai đoạn trùng nhau trong bản mã cách nhau một khoảng n ký tự, thì độ dài khóa có thể là một trong các ước số của n. Từ đó, bằng cách tìm nhiều cặp đoạn trùng nhau và lấy ước số chung lớn nhất (GCD) của các khoảng cách, ta có thể xác định được độ dài khóa tiềm năng.

Ví dụ, với bản rõ “ENGLISH OR SPANISH” và khóa là “KEY”, bản mã là “OREVMQR SP CTYXMQR”. Giả sử chúng ta chỉ có bản mã, chúng ta sẽ ước lượng và tìm ra độ dài khóa thông qua bản mã như sau:

“OREVMQRSPCTYXMQR”.

Tìm những đoạn giống nhau trong bản mã, ở đây có thể thấy là “MQR”

Pos	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Plantext	O	R	E	V	M	Q	R	S	P	C	T	Y	X	M	Q	R

- Vị trí của “MQR”$_1$ xuất hiện lần đầu tiên ở vị trí thứ 4 – 6

- Vị trí của “MQR”$_2$ xuất hiện lần thứ hai ở vị trí thứ 13 – 15

Khoảng cách giữa hai đoạn này là 6. Các ước số của 6 bao gồm 1, 2, 3, 6 và độ dài khóa có thể là 1 trong các ước số đó. Trong ví dụ, khóa là “KEY” có độ dài là 3.

Như vậy, phương pháp Kasiski cho phép chúng ta ước lượng độ dài khóa của mật mã Vigenère một cách tương đối chính xác. Việc xác định được độ dài khóa giúp thu hẹp đáng kể không gian tìm kiếm, tạo tiền đề cho các phương pháp phân tích tần suất và giải mã chi tiết ở các bước tiếp theo.

2.3 Ưu điểm và hạn chế

Một số ưu điểm của phương pháp Kasiski

- Đơn giản và dễ triển khai: chỉ cần dò tìm các đoạn lặp trong bản mã và tính khoảng cách giữa chúng.
- Hiệu quả với bản mã dài: khi bản mã đủ dài và có nhiều mẫu lặp, Kasiski thường nhanh chóng trả về ứng viên độ dài khóa chính xác.
- Thu hẹp không gian tìm kiếm: thay vì thử mọi độ dài, ta chỉ cần thử các ước của GCD thu được, tiết kiệm thời gian và tài nguyên cho bước phân tích tần suất.
- Làm tiền đề cho các bước tiếp theo: xác định k cho phép tách bản mã thành các cột tương ứng, từ đó áp dụng “Caesar-crack” cho từng cột để tìm khóa.

Một số nhược điểm và hạn chế

- Phụ thuộc vào độ dài và cấu trúc bản rõ: nếu bản mã ngắn hoặc bản rõ ít lặp, Kasiski dễ thất bại.
- Nhạy cảm với ký tự nhiễu: khoảng trắng, dấu câu hoặc các kí tự khác có thể làm sai vị trí và dẫn đến ước chung không chính xác.
- Không trả về khóa trực tiếp: Kasiski chỉ ước lượng độ dài khóa nên vẫn cần phân tích tần suất hoặc thử các ứng viên để tìm khóa thực.

- Có thể cho nhiều khóa ứng viên: GCD đôi khi trả về nhiều ước khả dĩ (hoặc 1), nên cần kết hợp thêm phương pháp để kiểm chứng.
- Độ tin cậy giảm khi mẫu ít: nếu chỉ tìm được ít đoạn lặp thì kết luận có thể sai, do đó cần nhiều mẫu để tăng độ chính xác.

CHƯƠNG 3. KỸ THUẬT PHÂN TÍCH TẦN SUẤT

3.1 Giới thiệu

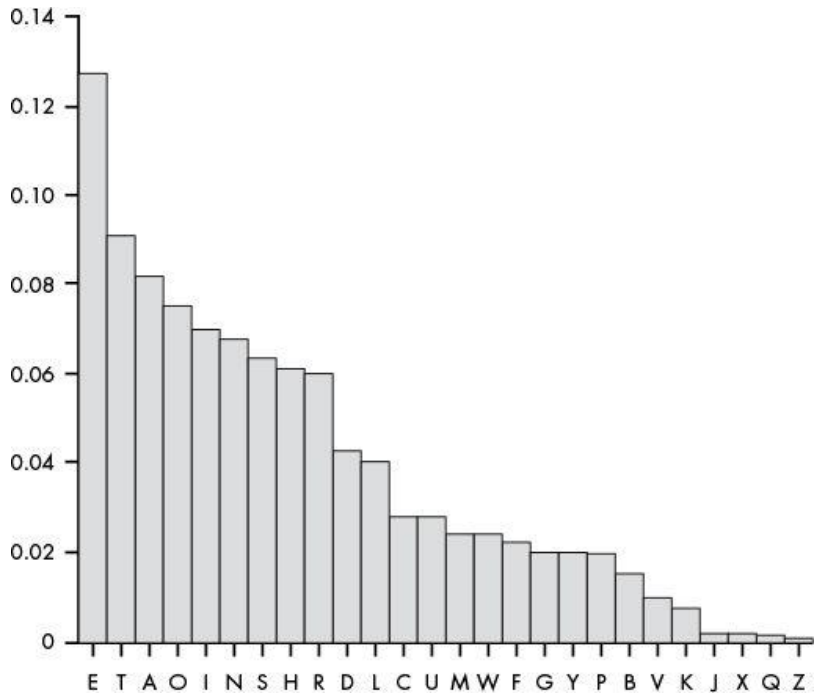
Phân tích tần suất (Frequency Analysis) là kỹ thuật cổ điển và nền tảng trong mật mã học, tận dụng sự bất đối xứng trong phân bố chữ cái của ngôn ngữ tự nhiên để suy luận thông tin. Khi đã biết (hoặc ước lượng) độ dài khóa m của mật mã Vigenère, ta có thể phân tách bản mã thành m chuỗi con độc lập, mỗi chuỗi tương ứng với một vị trí cố định trong khóa. Mỗi chuỗi con này về bản chất là kết quả của một Caesar cipher, nên có thể bị phá bằng các phương pháp phân tích tần suất riêng rẽ, từ đó suy ra từng ký tự của khóa và khôi phục bản rõ.

3.2 Cơ sở lý thuyết và triển khai

Trong các ngôn ngữ tự nhiên (chẳng hạn tiếng Anh), các chữ cái không xuất hiện với tần suất như nhau. Một số ký tự xuất hiện thường xuyên hơn hẳn so với phần còn lại, chẳng hạn trong tiếng Anh, các chữ cái E, T, A, O, I, N là những ký tự phổ biến nhất, chiếm tỷ lệ lớn trong hầu hết mọi văn bản. Chính sự mất cân bằng đặc trưng này trong tần suất xuất hiện đã tạo điều kiện cho các nhà phân tích lợi dụng để giải mã những hệ mật cổ điển như Caesar hay Vigenère.

Letter	English	French[21]	German[22]	Spanish[23]	Portuguese[24]	Italian[25]	Turkish[26]	Swedish[27]	Polish[28]	Dutch[29]	Danish[30]	Icelandic[31]	F
a	8.167%	7.636%	6.516%	11.525%	14.634%	11.745%	11.920%	9.383%	8.965%	7.49%	6.025%	10.110%	1
b	1.492%	0.901%	1.886%	2.215%	1.043%	0.927%	2.844%	1.535%	1.482%	1.58%	2.000%	1.043%	0
c	2.782%	3.260%	2.732%	4.019%	3.882%	4.501%	0.963%	1.486%	3.988%	1.24%	0.565%	~0%	0
d	4.253%	3.669%	5.076%	5.010%	4.992%	3.736%	4.706%	4.702%	3.293%	5.93%	5.858%	1.575%	1
e	12.702%	14.715%	16.396%	13.702%	13.101%	11.792%	8.912%	10.149%	7.921%	18.91%	15.453%	6.418%	7
f	2.228%	1.066%	1.656%	0.692%	1.023%	1.153%	0.461%	2.027%	0.312%	0.81%	2.406%	3.013%	0
g	2.015%	0.866%	3.009%	1.768%	1.303%	1.644%	1.253%	2.862%	1.377%	3.40%	4.077%	4.241%	0
h	6.094%	0.937%	4.577%	1.973%	1.281%	0.136%	1.212%	2.090%	1.072%	2.38%	1.621%	1.871%	1
i	6.966%	7.529%	6.550%	6.247%	6.186%	10.143%	8.600%*	5.817%	8.286%	6.50%	6.000%	7.578%	1
j	0.153%	0.813%	0.268%	0.493%	0.379%	0.011%	0.034%	0.614%	2.343%	1.46%	0.730%	1.144%	2
k	0.772%	0.074%	1.417%	0.026%	0.015%	0.009%	4.683%	3.140%	3.411%	2.25%	3.395%	3.314%	4
l	4.025%	5.456%	3.437%	4.967%	2.779%	6.510%	5.922%	5.275%	2.136%	3.57%	5.229%	4.532%	5
m	2.406%	2.968%	2.534%	3.157%	4.738%	2.512%	3.752%	3.471%	2.911%	2.21%	3.237%	4.041%	3
n	6.749%	7.095%	9.776%	6.712%	4.446%	6.883%	7.487%	8.542%	5.600%	10.03%	7.240%	7.711%	8
o	7.507%	5.796%	2.594%	8.683%	9.735%	9.832%	2.476%	4.482%	7.590%	6.06%	4.636%	2.166%	5
p	1.929%	2.521%	0.670%	2.510%	2.523%	3.056%	0.886%	1.839%	3.101%	1.57%	1.756%	0.789%	1
q	0.095%	1.362%	0.018%	0.877%	1.204%	0.505%	0	0.020%	0.003%	0.009%	0.007%	0	0
r	5.987%	6.693%	7.003%	6.871%	6.530%	6.367%	6.722%	8.431%	4.571%	6.41%	8.956%	8.581%	2
s	6.327%	7.948%	7.270%	7.977%	6.805%	4.981%	3.014%	6.590%	4.263%	3.73%	5.805%	5.630%	7
t	9.056%	7.244%	6.154%	4.632%	4.336%	5.623%	3.314%	7.691%	3.966%	6.79%	6.862%	4.953%	6
u	2.758%	6.311%	4.166%	3.927%	3.639%	2.813%	3.235%	1.919%	2.347%	1.99%	1.979%	4.562%	5
v	0.978%	1.838%	0.846%	1.138%	1.575%	2.097%	0.959%	2.415%	0.034%	2.85%	2.332%	2.437%	2
w	2.360%	0.049%	1.921%	0.027%	0.037%	0.033%	0	0.142%	4.549%	1.52%	0.069%	0	0

Hình 3.1 Tần suất xuất hiện của các chữ cái trong một số ngôn ngữ tự nhiên



Hình 3.2 Tần suất các chữ cái xuất hiện (Aphalbet) giảm dần

Dựa trên cơ sở đó, các nhà phân tích mật mã đã triển khai phương pháp phân tích tần suất đối với mã hóa Vigenère như sau:

1. Sau khi ước lượng được độ dài khóa, bản mã chia thành nhiều chuỗi bản mã con tương ứng với một vị trí trong khóa.
2. Mỗi chuỗi con này về bản chất là một mật mã Caesar độc lập, do đó có thể được phân tích dựa trên phân bố tần suất chữ cái. Bằng cách so sánh tần suất xuất hiện trong chuỗi con với tần suất chuẩn của ngôn ngữ, ta có thể xác định được độ dịch chuyển – tức là ký tự khóa tương ứng cho mỗi vị trí, từ đó tái tạo toàn bộ khóa và giải mã bản mã.
3. Sau khi phân tích xong và giải mã các bản mã con, tiến hành tổng hợp lại các chuỗi kết quả con đã được giải mã. Từ đó cho ra kết quả cuối cùng của việc giải mã bản mã gốc

3.3 Kiểm định Chi – Bình phương (Chi-squared Test)

Kiểm định Chi-bình phương (Chi-squared Test) là một kỹ thuật bổ trợ quan trọng trong phân tích tần suất. Về lý thuyết, phương pháp phân tích tần suất dựa trên việc so sánh sự xuất hiện của các ký tự trong bản mã với tần suất thực tế của ngôn ngữ (như tiếng Anh). Tuy nhiên, cách làm này chỉ mang tính ước lượng và đôi khi thiếu độ chính xác cần thiết.

Chính vì vậy, Kiểm định Chi-bình phương được sử dụng như một công cụ thống kê giúp đánh giá mức độ tương đồng giữa phân bố ký tự trong bản mã và phân bố ngôn ngữ gốc. Nhờ đó, ta có thể định lượng mức sai lệch, từ đó xác định khóa khả dĩ chính xác hơn, làm tăng đáng kể độ tin cậy của kết quả giải mã.

3.3.1 Cơ sở lý thuyết

Phương pháp này dựa trên xác suất thống kê, cụ thể là kiểm định độ phù hợp (Goodness-of-fit test). Mục tiêu của nó là đo xem phân bố tần suất của dữ liệu quan sát (observed) khác bao nhiêu so với phân bố kỳ vọng (expected) trong một mô hình lý thuyết, ở đây chính là phân bố tần suất ký tự trong ngôn ngữ tự nhiên (ví dụ tiếng Anh).

Nói cách khác, kiểm định Chi-bình phương giúp chúng ta đánh giá xem bản mã đã được giải thử bằng một khóa nào đó có giống ngôn ngữ gốc đến mức nào. Nếu phân bố tần suất sau khi giải thử càng gần với tần suất tiếng Anh chuẩn, khóa đó càng có khả năng đúng.

Công thức:

$$x^2 = \sum_{i=0}^{n-1} \frac{(O_i - E_i)^2}{E_i}$$

Trong đó:

- O_i là tần suất quan sát được của ký tự thứ i trong bản mã sau khi giải thử bằng khóa nào đó
- E_i là tần suất kỳ vọng của ký tự đó trong ngôn ngữ thật (ví dụ Tiếng Anh)

- n là số lượng ký tự xét

Chúng ta sẽ sử dụng tần suất ký tự trong bản chữ cái Alphabet làm ví dụ cho các bước triển khai tiếp theo, với các ngôn ngữ khác, nó sẽ có các tần suất ký tự khác nhau. Trong nội dung chương này, chúng ta sử dụng Tiếng Anh làm chuẩn và bản tần suất ký tự trong ngôn ngữ Tiếng Anh bên dưới.

Chữ cái	**Tần suất (%)**	**Dạng thập phân (p_i)**
E	12.70%	0.12702
T	9.06%	0.09056
A	8.17%	0.08167
O	7.51%	0.07507
I	6.97%	0.06966
N	6.75%	0.06749
S	6.33%	0.06327
H	6.09%	0.06094
R	5.99%	0.05987
D	4.25%	0.04253
L	4.03%	0.04025
C	2.78%	0.02782
U	2.76%	0.02758
M	2.41%	0.02406
W	2.36%	0.0236
F	2.23%	0.02228
G	2.02%	0.02015
Y	1.97%	0.01974
P	1.93%	0.01929
B	1.49%	0.01492
V	0.98%	0.00978
K	0.77%	0.00772

J	0.15%	0.00153
X	0.15%	0.0015
Q	0.10%	0.00095
Z	0.07%	0.00074

Bảng 1 Bảng tần suất ký tự trong Tiếng Anh

3.3.2 Các bước triển khai

Giả sử chúng ta có khóa K có độ dài là n và tập $\{K_1, K_2, \dots, K_n\}$ với K_m tương ứng với một ký tự trong khóa K. Chúng ta thực hiện phép dịch khóa (26 lần): với mỗi nhóm K_m lần lượt dịch chuỗi theo 26 giá trị, mỗi lần dịch sẽ cho một bản rõ giả định, ta sẽ tính Chi bình phương tương ứng như sau:

- Tính số lần xuất hiện của mỗi ký tự thứ i (O_i) bằng cách đếm số lần xuất hiện của mỗi ký tự tương ứng trong nhóm K_m
- Tính tần suất kỳ vọng mỗi ký tự thứ i (E_i):

$$E_i = p_i \times N_m$$

  Trong đó:

  - p_i là tần suất xuất hiện ký tự thứ i trong bảng chữ cái
  - N_m là số lượng ký tự trong nhóm K_m
- Tính chi – bình phương x^2 theo công thức:

$$x^2 = \sum_{i=0}^{n-1} \frac{(O_i - E_i)^2}{E_i}$$

Sau khi tính xong, chúng ta sẽ có một danh sách các x^2 tương ứng cho 26 ký tự khóa mà chúng ta đã dịch. Ký tự có khả năng cao là khóa là ký tự có x^2 nhỏ nhất.

CHƯƠNG 4. THỰC HIỆN GIẢI MÃ CHI TIẾT

4.1 Thực hiện giải mã thủ công

Chúng ta sẽ sử dụng bản mã dưới đây

PeogrzqngcgbxerqunrNxxekiogpaegtrquwqwgPeogrzqngcrnpakqugfagfdxuekvyhwNxxekiogpafckxaonerrPeogrzqngcstaghqduaeGgvxtghpntveenaeketpdawrmaon

4.1.1 Ước lượng độ dài khóa với Kasiski

Trước khi đi vào phương pháp Kasiski, ta sẽ quy định chuỗi lặp lại có độ dài tối thiểu là 10. Sử dụng phương pháp Kasiski, ta tìm được hai chuỗi lặp lại trong bản mã là "Peogrzqngc" (3 lần lặp lại) và "Nxxekiogpa" (2 lần lặp lại)

Peogrzqngcgbxerqunr Nxxekiogpa egtrquwqwg Peogrzqngc rnpakqugfagfdxuekvyhw Nxxekiogpa fckxaonerr Peogrzqngc staghqduaeGgvxtghpntveenaeketpdawrmaon

Với chuỗi "Nxxekiogpa", hai chuỗi lặp lại cách nhau 51 ký tự. Các ước số của 51 là {1, 3, 17, 51}

Với chuỗi "Peogrzqngc", chuỗi này lặp lại 3 lần. Ta tính khoảng cách giữa chuỗi xuất hiện lần đầu tiên với các xuất hiện tiếp theo của nó, cụ thể:

- "$Peogrzqngc_1$" đến "$Peogrzqngc_2$": 39 ký tự. Các ước số là {1, 3, 13, 39}
- "$Peogrzqngc_1$" đến "$Peogrzqngc_3$": 51 ký tự. Các ước số là {1, 3, 17, 51}

Từ hai thông tin trên, ta có thể thu hẹp các giá trị ước số của hai chuỗi, tập trung vào những ước vừa xuất hiện nhiều nhất vừa lớn hơn 1. Trong trường hợp này, ước số 3 là ước ứng viên hợp lý nhất cho độ dài khóa tiềm năng.

4.1.2 Thực hiện phân tích tần suất với bản mã (độ dài khóa là 3)

Với khóa có độ dài là 3, ta gọi khóa K có dạng $K_1K_2K_3$ với K_i đại diện cho từng kí tự khóa. Khi đó ta sẽ nhóm lại thành 3 nhóm tương ứng như sau:

P	e	o	g	r	z	q	n	c	…	a	w	r	m	a	o	n
K_1	K_2	K_3	K_1	K_2	K_3	K_1	K_2	K_3	…	K_3	K_1	K_2	K_3	K_1	K_2	K_3

Giải thích: Bản mã có độ dài là 138 ký nên khóa sẽ lấp đầy mà không thiếu

Chia làm 3 nhóm K_1, K_2, K_3 (mỗi nhóm 46 ký tự):

- K_1 : "PgqcxqrxipgqqPgqcpqffuvwxipcaePgqcaqagtpvnkpwa"
- K_2: "erngeuNeoatuwernrauadeyNeoakorernsgdevgneaedro"
- K_3: "ozgbrnxkgerwgozgnkggxkhxkgfxnrozgthuGxhteetamn"

Bước tiếp theo chúng ta thực hiện kiểm định Chi bình phương cho mỗi nhóm K_1, K_2, K_3

4.1.2.1 Giải mã K_1

K_1 : "PgqcxqrxipgqqPgqcpqffuvwxipcaePgqcaqagtpvnkpwa"

Chúng ta có $N_1 = 46$. Tính tuần suất kỳ vọng E_i dựa vào bảng tần suất ký tự ở trên

Ký tự	p_i	E_i
A	0.08167	3.75682
B	0.01492	0.68632
C	0.02782	1.27972
D	0.04253	1.95638
E	0.12702	5.84292
F	0.02228	1.02488
G	0.02015	0.9269
H	0.06094	2.80324
I	0.06966	3.20436
J	0.00153	0.07038
K	0.00772	0.35512
L	0.04025	1.8515

M	0.02406	1.10676
N	0.06749	3.10454
O	0.07507	3.45322
P	0.01929	0.88734
Q	0.00095	0.0437
R	0.05987	2.75402
S	0.06327	2.91042
T	0.09056	4.16576
U	0.02758	1.26868
V	0.00978	0.44988
W	0.0236	1.0856
X	0.0015	0.069
Y	0.01974	0.90804
Z	0.00074	0.03404

Bảng 2 Kết quả tính O_i ở K_1

Bước tiếp theo chúng ta sẽ thực hiện phép giải mã dịch trái lên K_1, mỗi lần dịch ta sẽ tính và có một bảng gồm Các O_i. Như vậy sau khi dịch 26 lần tương ứng 26 ký tự chúng ta sẽ có 26 bảng O_i !

Shift	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0
1	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4
2	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0
3	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4
4	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0
5	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1
6	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2
7	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5
8	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0
9	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2
10	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0
11	0	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1
12	0	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0
13	1	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0
14	0	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1
15	8	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0
16	8	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8
17	1	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8
18	0	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1
19	1	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0
20	1	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1
21	2	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1
22	2	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2
23	3	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2
24	0	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3
25	0	4	0	4	0	1	2	5	0	2	0	1	0	0	1	0	8	8	1	0	1	1	2	2	3	0

Bảng 3 Kết quả sau khi tính O_i ở mọi lần dịch (K_1)

Từ dữ liệu bảng 2 và bảng 3, chúng ta áp dụng công thức tính Chi bình phương:

$$x^2 = \sum_{i=1}^{n} \frac{(O_i - E_i)^2}{E_i}$$

Cuối cùng, kết quả Chi bình phương sau khi giải mã thử là:

Shift	Ký tự	x^2

0	A	1687.252
1	B	623.9359
2	C	53.25165
3	D	791.7618
4	E	149.6729
5	F	604.8345
6	G	1300.298
7	H	1899.131
8	I	132.3996
9	J	554.0684
10	K	521.0689
11	L	235.4889
12	M	470.2883
13	N	176.331
14	O	315.8464
15	P	194.583
16	Q	2517.791
17	R	2188.725
18	S	1148.723
19	T	1200.936
20	U	287.0052
21	V	255.0844
22	W	321.125
23	X	537.8399
24	Y	368.2263
25	Z	1620.055

Bảng 4 Kết quả Chi bình phương ở K_1

Có thể nhận thấy rằng shift = 2, tương ứng với ký tự khóa “C”, cho ra giá trị Chi bình phương nhỏ nhất là 53.25165 , điều này cho thấy “C” rất có khả năng là ký tự khóa đúng cho vị trí đang xét. Kết luận: ký tự khóa tiềm năng của K_1 là “C”

4.1.2.2 Giải mã K_2

K_2: “erngeuNeoatuwernrauadeyNeoakorernsgdevgneaedro”

Các bước thực hiện tương tự với giải mã ký tự khóa K_1, do là độ dài nhóm K_2 bằng với K_1 ($N_2 = 46$) vì thế ta sử dụng lại Bảng 2. Bước tiếp theo là thực hiện phép giải mã dịch trái với K_2 và tính các O_i

Shift	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0
1	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5
2	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0
3	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0
4	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3
5	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10
6	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0
7	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3
8	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0
9	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0
10	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0
11	0	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1
12	0	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0
13	6	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0
14	4	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6
15	0	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4
16	0	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0
17	6	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0
18	1	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6
19	1	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1
20	3	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1
21	1	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3
22	1	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1
23	0	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1
24	1	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0
25	0	5	0	0	3	10	0	3	0	0	0	1	0	0	6	4	0	0	6	1	1	3	1	1	0	1

Bảng 5 Kết quả sau khi tính O_i ở mọi lần dịch (K_2)

Kết quả Chi bình phương sau khi giải mã thử ở K_2 là

Shift	Ký tự	x^2
0	A	36.11885
1	B	1651.418
2	C	177.3984
3	D	619.9013
4	E	1053.174
5	F	3276.631
6	G	282.2989
7	H	1834.516
8	I	684.2823
9	J	381.5901
10	K	691.2004
11	L	266.0266
12	M	129.3165
13	N	280.3482
14	O	3373.655
15	P	651.2157
16	Q	860.0063
17	R	642.91
18	S	1222.822
19	T	194.1456
20	U	999.7348
21	V	1739.642
22	W	172.6702
23	X	1163.859
24	Y	536.4222
25	Z	199.9396

Bảng 6 Kết quả Chi bình phương ở K_2

Dựa vào kết quả trên, shift = 0 tương ứng với ký tự khóa “A” có Chi bình phương nhỏ nhất là 36.11885, điều này cho thấy “A” rất có khả năng là ký tự khóa đúng cho vị trí đang xét. Kết luận: ký tự khóa tiềm năng của K_2 là “A”

4.1.2.3 Giải mã K_3

K_3: “ozgbrnxkgerwgozgnkggxkhxkgfxnrozgthuGxhteetamn”

Thực hiện tương tự với giải mã K_1 và K_2. Do độ dài nhóm K_3 bằng với K_1 ($N_3 = 46$) nên ta sử dụng lại Bảng 2. Bước tiếp theo là thực hiện phép giải mã dịch trái với K_3 và tính các O_i

Shift	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3
1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1
2	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1
3	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0
4	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0
5	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3
6	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1
7	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9
8	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3
9	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0
10	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0
11	0	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4
12	1	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0
13	4	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1
14	3	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4
15	0	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3
16	0	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0
17	3	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0
18	0	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3
19	3	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0
20	1	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3
21	0	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1
22	1	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0
23	5	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1
24	0	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5
25	3	1	1	0	0	3	1	9	3	0	0	4	0	1	4	3	0	0	3	0	3	1	0	1	5	0

Bảng 7 Kết quả sau khi tính O_i ở mọi lần dịch (K_3)

Kết quả Chi bình phương sau khi giải mã thử ở K_3 là

Shift	Ký tự	x^2
0	A	736.4077
1	B	566.8359
2	C	239.7966
3	D	333.4191
4	E	373.6915

5	F	523.6088
6	G	85.47791
7	H	3108.239
8	I	534.3328
9	J	1437.005
10	K	359.3724
11	L	710.3541
12	M	122.3534
13	N	367.7406
14	O	1054.648
15	P	396.097
16	Q	2228.991
17	R	463.3788
18	S	366.7788
19	T	56.06026
20	U	852.2529
21	V	234.7919
22	W	434.3692
23	X	1583.066
24	Y	1128.973
25	Z	78.37134

Bảng 8 Kết quả Chi bình phương ở K_3

Dựa vào kết quả trên, shift = 19 tương ứng với ký tự khóa "T" có Chi bình phương nhỏ nhất là 56.06026, điều này cho thấy "T" rất có khả năng là ký tự khóa đúng cho vị trí đang xét. Kết luận: ký tự khóa tiềm năng của K_3 là "T".

4.1.3 Tập hợp khóa và giải mã bản mã

Sau khi phân tích tần suất với bản mã, chúng ta nhận được 3 ký tự khóa tiềm năng (khóa có khả năng là khóa đúng) gồm:

- K_1 : “C”
- K_2 : “A”
- K_3 : “T”

Từ đó, suy ra khóa tiềm năng là “CAT”.

Quay lại bản mã, tiếp theo là giải mã bản mã với khóa tiềm năng đã phân tích

- **Bản mã:**
  “PeogrzqngcgbxerqunrNxxekiogpaegtrquwqwgPeogrzqngcrnpakqugfagfdxuekvyhwNxxekiogpafckxaonerrPeogrzqngcstaghqduaeGgvxtghpntveenaeketpdawrmaon”
- **Khóa:** “CAT”

P	e	o	g	r	z	q	n	c	…	a	w	r	m	a	o	n
C	A	T	C	A	T	C	A	T	…	T	C	A	T	C	A	T
N	e	v	e	r	g	o	n	n		h	u	r	t	y	o	u

- **Bản rõ:**
  “NevergonnagiveyouupNevergonnaletyoudownNevergonnarunaroundanddesertyouNevergonnamakeyoucryNevergonnasaygoodbyeNevergonnatellalieandhurtyou”

Như vậy là chúng ta đã giải mã thành công bản mã trên, thông điệp của bản rõ được giải mã là lời bài hát Never gonna give you up của Rick Astley.

“Never gonna give you up

Never gonna let you down

Never gonna run around and desert you

Never gonna make you cry

Never gonna say goodbye

Never gonna tell a lie and hurt you”

4.2 Giải mã bằng chương trình

4.2.1 Giới thiệu và cấu trúc tổng thể của chương trình

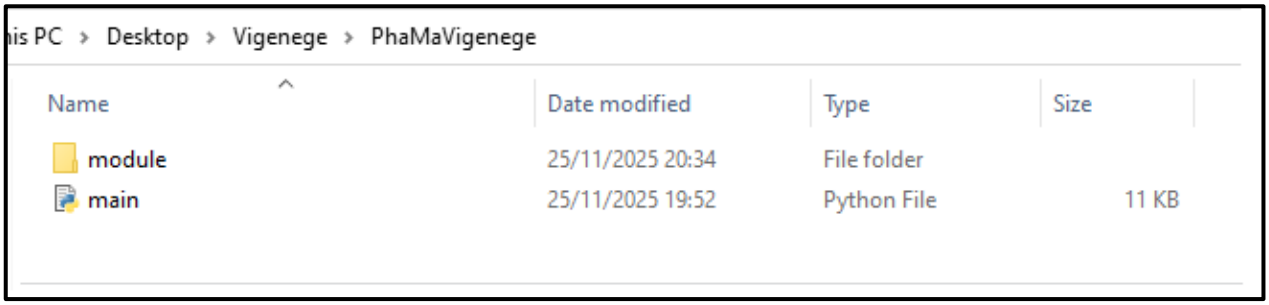
Trong mục này, chương trình giải mã được sử dụng như một công cụ tự động nhằm hỗ trợ quá trình phân tích và phá mã Vigenère. Các thao tác vốn phải xử lý thủ công như nhận diện chuỗi lặp, xác định khoảng cách giữa các vị trí xuất hiện hay suy đoán khóa đều được chương trình thực hiện một cách có hệ thống. Nhờ vậy, việc giải mã trở nên nhanh chóng và ổn định hơnm đặc biệt hữu ích khi làm việc với bản mã có độ dài lớn hoặc cấu trúc phức tạp hơn.

Chương trình áp dụng hai phương pháp bao gồm: Phương pháp Kasiski dùng để ước lượng độ dài khóa và phương pháp phân tích tần suất sử dụng Chi bình phương.

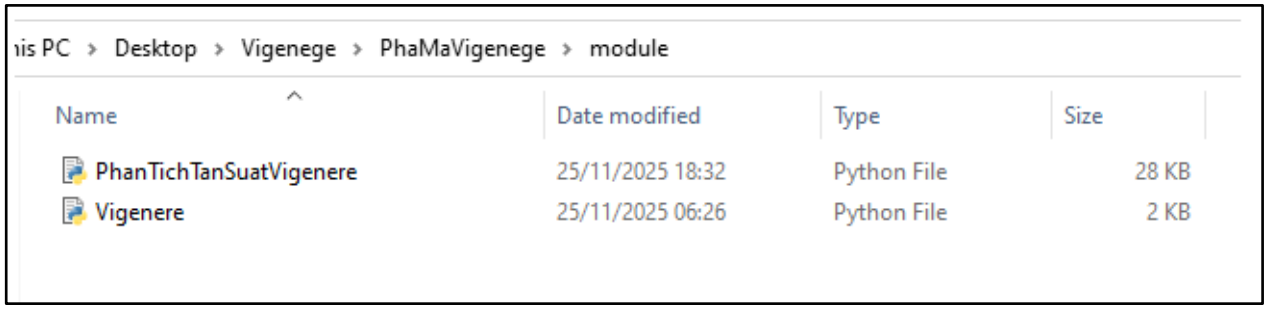
4.2.2 Cấu trúc tổng thể của chương trình

Tổ chức chương trình bao gồm:

- Main.py: file chương trình chính, đây là file khởi động đầu tiên
- Thư mục module gồm hai module được viết bao gồm:
  + Vigenere.py: module dùng để mã hóa và giải mã mã hóa Vigenère
  + PhanTichTanSuatVigenere.py: module phục vụ cho việc phân tích tần suất trong trường hợp không biết khóa



Hình 4.1 Thư mục chương trình PhaMaVigenege



Hình 4.2 Bên trong thư mục module

Chương trình được thiết kế theo dạng các module độc lập, trong đó mỗi module đảm nhiệm một chức năng cụ thể của quá trình phân tích và giải mã Vigenère. Cách tổ chức này giúp chương trình dễ bảo trì, dễ mở rộng và đảm bảo rõ ràng trong luồng xử lý. Toàn bộ hệ thống có thể được chia thành các khối chức năng sau:

1. **Khối tìm chuỗi lặp (Repeated Sequence Detection):** Khối này chịu trách nhiệm quét toàn bộ ciphertext để tìm các chuỗi ký tự lặp lại. Những chuỗi lặp này là dữ liệu đầu vào quan trọng cho bước tính khoảng cách.

```
def find_repeated_sequences(ciphertext, min_length=3):
    text = ''.join(c for c in ciphertext.upper() if c.isalpha())
    repeated = {}
    for length in range(min(len(text) // 2, 20), min_length - 1, -1):
        seen = {}
        for i in range(len(text) - length + 1):
            seq = text[i:i + length]
            if seq in seen:
                repeated.setdefault(seq, set()).add(seen[seq])
                repeated[seq].add(i)
            else:
                seen[seq] = i
    result = {
        seq: sorted(pos)
```

```
        for seq, pos in repeated.items()
        if len(pos) >= 2
    }
    return result
```

Hình 4.3 Hàm find_repeated_sequences

2. **Khối tính khoảng cách và suy đoán độ dài khóa (Kasiski Examination):** Từ các vị trí xuất hiện của chuỗi lặp, chương trình tính khoảng cách và trích xuất các ước số chung. Các ước số này được dùng để đề xuất các chiều dài khóa khả thi.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def gcd_list(numbers):
    if not numbers:
        return 1
    result = numbers[0]
    for num in numbers[1:]:
        result = gcd(result, num)
    return result

def get_factors(n, max_factor=30):
    factors = []
    for i in range(2, min(n + 1, max_factor + 1)):
        if n % i == 0:
            factors.append(i)
    return factors
```

Hình 4.4 Các hàm phục vụ cho việc tính ước chung

```
def kasiski_analysis(ciphertext, min_seq_length=3):
    seqs = find_repeated_sequences(ciphertext,
min_length=min_seq_length)

    if not seqs:
        return None, []
    all_distances = []
    sequence_info = []

    sorted_seqs = sorted(seqs.items(), key=lambda x: (-len(x[0]), -
len(x[1])))

    for seq, positions in sorted_seqs:
        distances = []
        for i in range(len(positions) - 1):
            dist = positions[i + 1] - positions[i]
            distances.append(dist)
            all_distances.append(dist)

        factors = set()
        for d in distances:
            factors.update(get_factors(d))

        sequence_info.append({
            "sequence": seq,
            "length": len(seq),
            "positions": positions,
            "count": len(positions),
            "distances": distances,
            "factors": sorted(factors),
        })
```

```
    likely_lengths = []
    if all_distances:
        factor_count = Counter()
        for d in all_distances:
            for f in get_factors(d):
                factor_count[f] += 1
        likely_lengths = [f for f, _ in factor_count.most_common(15)]

    if not likely_lengths:
        likely_lengths = list(range(1, 11))

    return sequence_info, likely_lengths
```

Hình 4.5 Hàm kasiski_analysis

3. **Khối phân tích tần suất ký tự (Frequency Analysis):** Sau khi xác định độ dài khóa, khối này chia ciphertext thành các nhóm ký tự theo modulo và phân tích tần suất từng nhóm để xác định từng ký tự khóa.

```
def analyze_key_length(ciphertext, key_length):
    text = ''.join(c for c in ciphertext.upper() if c.isalpha())
    columns = [''] * key_length
    for i, ch in enumerate(text):
        columns[i % key_length] += ch

    key_chars = []
    column_results = []

    for col_idx, col in enumerate(columns):
        best_shift = 0
        best_chi = float('inf')
        all_shifts = []
        for shift in range(26):
```

```
            dec = decrypt_caesar(col, shift)
            chi = chi_squared_test(dec, FREQ_ENGLISH)
            all_shifts.append({
                "shift": shift,
                "letter": chr(shift + ord('A')),
                "chi": chi,
                "preview": dec[:50]
            })
            if chi < best_chi:
                best_chi = chi
                best_shift = shift

        key_char = chr(best_shift + ord('A'))
        key_chars.append(key_char)
        column_results.append({
            "column_index": col_idx,
            "best_shift": best_shift,
            "best_letter": key_char,
            "best_chi": best_chi,
            "all_shifts": all_shifts
        })

    key = ''.join(key_chars)
    plaintext = decrypt_full(ciphertext, key)

    letters_only = ''.join(c for c in plaintext.upper() if c.isalpha())
    chi_total = chi_squared_test(letters_only, FREQ_ENGLISH)
    cw_count = count_common_words(plaintext)
    cw_list = list_common_words_in_text(plaintext)

    score = (1 / (chi_total + 1)) * 100 + cw_count * 5
```

```
    return {
        "key_length": key_length,
        "key": key,
        "plaintext": plaintext,
        "chi_squared": chi_total,
        "common_words": cw_count,
        "common_words_list": cw_list,
        "score": score,
        "column_results": column_results,
    }
```

Hình 4.6 Hàm analyze_key_length phân tích độ dài khóa cho trước

```
def chi_squared_test(text, freq_expected):
    text = ''.join(c for c in text.upper() if c.isalpha())
    total = len(text)
    if total == 0:
        return float('inf')
    freq_obs = Counter(text)
    chi = 0.0
    for letter in 'ABCDEFGHIJKLMNOPQRSTUVWXYZ':
        observed = freq_obs.get(letter, 0)
        expected = (freq_expected[letter] / 100.0) * total
        if expected > 0:
            chi += (observed - expected) ** 2 / expected
    return chi
```

Hình 4.7 Hàm chi_squared_test để tính Chi bình phương

4. **Khối giải mã văn bản (Vigenère Decryption Module):** Dựa trên khóa đã suy luận, khối này tiến hành giải mã toàn bộ ciphertext để thu được plaintext.

```
def decrypt_full(ciphertext, key):
    key = key.upper()
```

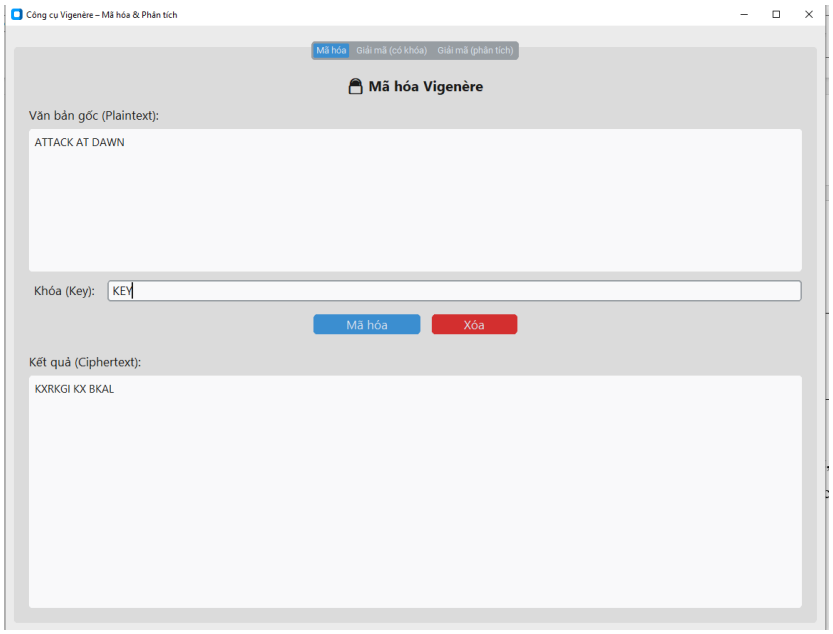
```
    result = []
    idx = 0
    for ch in ciphertext:
        if ch.isalpha():
            shift = ord(key[idx % len(key)]) - ord('A')
            if ch.isupper():
                base = ord('A')
                dec = chr((ord(ch) - base - shift) % 26 + base)
            else:
                base = ord('a')
                dec = chr((ord(ch) - base - shift) % 26 + base)
            result.append(dec)
            idx += 1
        else:
            result.append(ch)
    return ''.join(result)
```

Hình 4.8 Hàm decrypt_full

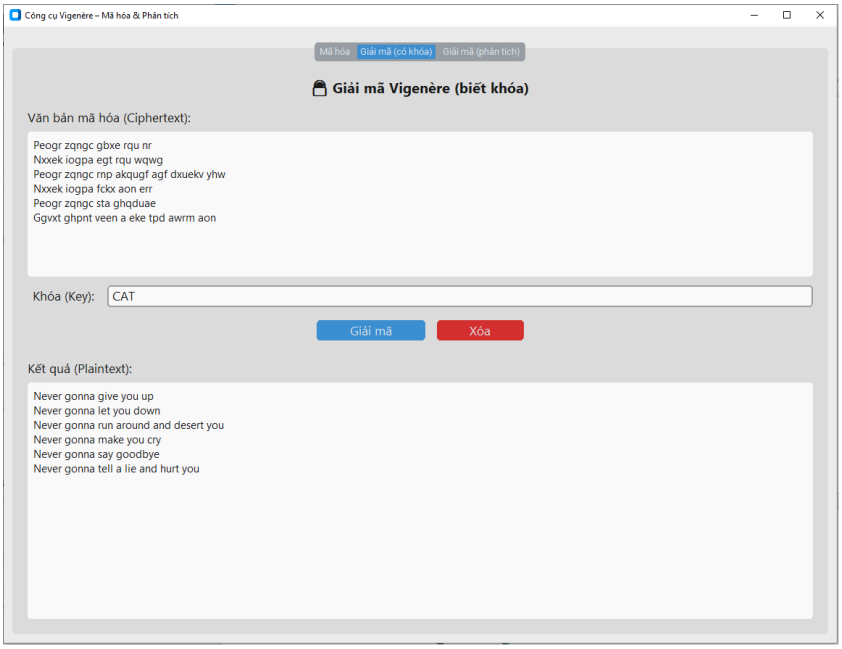
```
def decrypt_caesar(ciphertext, shift):
    out = []
    for c in ciphertext.upper():
        if c.isalpha():
            base = ord('A')
            out.append(chr((ord(c) - base - shift) % 26 + base))
        else:
            out.append(c)
    return ''.join(out)
```

Hình 4.9 Hàm decrypt_caesar

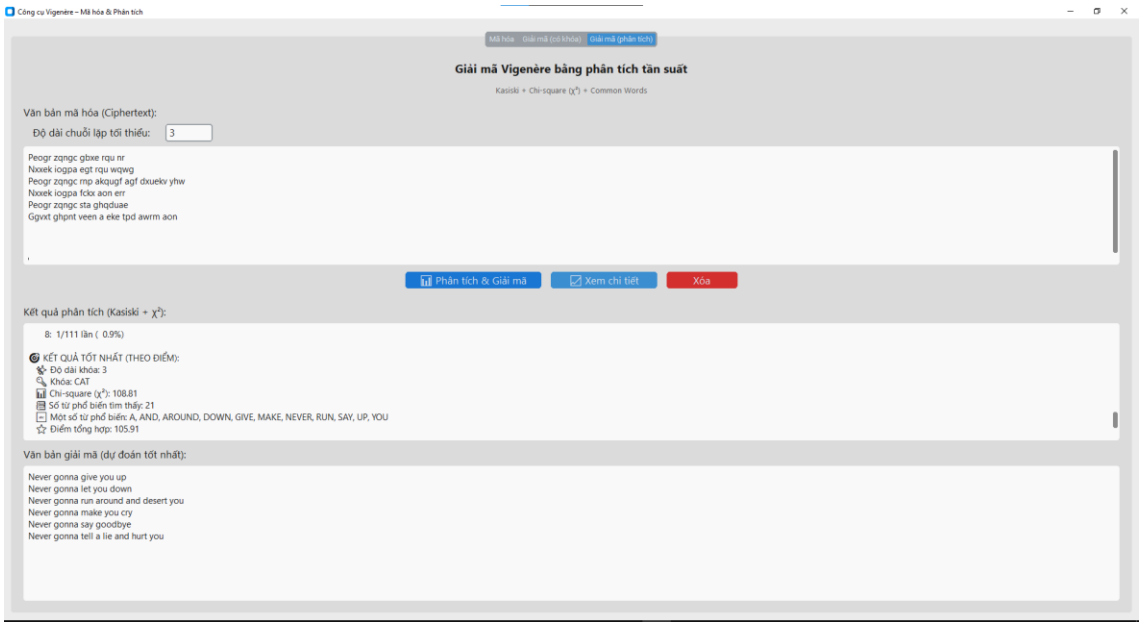
5. **Khối nhập/xuất (Input/Output Interface):** Cho phép người dùng nhập ciphertext, hiển thị các khóa khả thi, chọn khóa và xem kết quả giải mã. Khối nhập xuất được chia làm 3 phần.



Hình 4.10 Mã hóa Vigenère



Hình 4.11 Giải mã Vigenère khi biết được khóa



Hình 4.12 Giải mã Vigenère khi chưa biết khóa bằng phân tích tần suất

Đối với mã hóa và giải mã vigenère thông thường (tức biết khóa), khối giải mã chỉ bao gồm 2 hàm mã hóa và giải mã nằm trong module Vigenère.py

```
def encrypt_vigenere(plaintext: str, key: str) -> str:
    if not key or not key.isalpha():
        raise ValueError("Key phải là chuỗi chữ cái không rỗng")

    key = key.upper()
    ciphertext = ""
    key_index = 0

    for char in plaintext:
        if char.isalpha():
            is_upper = char.isupper()
            char_upper = char.upper()

            shift = ord(key[key_index % len(key)]) - ord('A')
            encrypted_char = chr((ord(char_upper) - ord('A') + shift) % 26
+ ord('A'))

            if is_upper:
                ciphertext += encrypted_char
            else:
                ciphertext += encrypted_char.lower()

            key_index += 1
```

```
        else:
            ciphertext += char

    return ciphertext


def decrypt_vigenere(ciphertext: str, key: str) -> str:

    if not key or not key.isalpha():
        raise ValueError("Key phải là chuỗi chữ cái không rỗng")

    key = key.upper()
    plaintext = ""
    key_index = 0

    for char in ciphertext:
        if char.isalpha():
            is_upper = char.isupper()
            char_upper = char.upper()

            shift = ord(key[key_index % len(key)]) - ord('A')
            decrypted_char = chr((ord(char_upper) - ord('A') - shift) % 26
+ ord('A'))

            if is_upper:
                plaintext += decrypted_char
            else:
                plaintext += decrypted_char.lower()

            key_index += 1
        else:
            plaintext += char

    return plaintext
```

Hình 4.13 Hàm encrypt_vigenere() và decrypt_vigenere ()

Ngoài ra, chương trình còn được mở rộng thêm hai chức năng quan trọng nhằm tăng độ chính xác trong việc lựa chọn khóa. Thứ nhất, hệ thống thực hiện tính toán giá trị Chi bình phương sau khi đã giải mã giúp so sánh mức độ phù hợp của từng khóa với phân bố tần suất tiếng Anh. Thứ hai, chương trình kiểm tra danh sách các từ tiếng Anh xuất hiện trong plaintext, từ đó đánh giá mức "tự nhiên" của văn bản sau giải mã.

Hai thông số này được kết hợp lại thành một điểm số tổng hợp (score). Score càng cao thì khả năng khóa đó là khóa đúng càng lớn. Nhờ cơ chế chấm điểm này, chương trình có thể lựa chọn khóa tối ưu một cách tự động và chính xác hơn so với việc chỉ dựa vào Kasiski hoặc phân tích tần suất riêng lẻ.

```
27 COMMON_WORDS = [
28 'THE','BE','TO','OF','AND','A','IN','THAT','HAVE','I','IT','FOR','NOT','ON','WITH','HE','AS','YOU','DO','AT',
29 'THIS','BUT','HIS','BY','FROM','THEY','WE','SAY','HER','SHE','OR','AN','WILL','MY','ONE','ALL','WOULD','THERE','THEIR','WHAT
30 'SO','UP','OUT','IF','ABOUT','WHO','GET','WHICH','GO','ME','WHEN','MAKE','CAN','LIKE','TIME','NO','JUST','HIM','KNOW','TAKE'
31 'PEOPLE','INTO','YEAR','YOUR','GOOD','SOME','COULD','THEM','SEE','OTHER','THAN','THEN','NOW','LOOK','ONLY','COME','ITS','OVE
32 'BACK','AFTER','USE','TWO','HOW','OUR','WORK','FIRST',' WELL','WAY','EVEN','NEW','WANT','BECAUSE','ANY','THESE','GIVE','DAY'
33 'AM','WAS','ARE','WERE','HAS','HAD','DID','MADE','SAID','MANY','LONG','OWN','DOWN','OFF','GOT','LEFT','RIGHT','EVERY','NEVER
34 'MUST','SHALL','SHOULD','MAY','MIGHT','YOUNG','OLD','LARGE','SMALL','LITTLE','BIG','GREAT','HIGH','LOW','EARLY','LATE','ABLE
35 'BETWEEN','BELOW','ABOVE','UNDER','AGAIN','AGAINST','AROUND','AFTERNOON','ALMOST','ALREADY','ALONG','ALWAYS','ANYONE','ANYTH
36 'ASK','ANSWER','HELP','TRY','FIND','FEEL','BELIEVE','SEEM','APPEAR','BEGIN','START','STOP','KEEP','HOLD','BRING','CARRY','MO
37 'EAT','DRINK','SLEEP','READ','WRITE','WATCH','LISTEN','SPEAK','TALK','CALL','PAY','BUY','SELL','SEND','RECEIVE','OPEN','CLOS
38 'RUN','WALK','TURN','CHANGE','CONTINUE','LEARN','STUDY','TEACH','UNDERSTAND','REMEMBER','FORGET','DECIDE','CHOOSE','PROVIDE'
39 'PLAY','WORK','DRIVE','FLY','TRAVEL','MEET','VISIT','RETURN','STAY','WAIT','HOPE','PLAN','AGREE','OFFER','ORDER','NEED','WIS
40 'HAPPY','SAD','ANGRY','TIRED','READY','IMPORTANT','INTERESTING','USEFUL','DIFFERENT','SIMILAR','TRUE','FALSE','REAL','FACT',
41 'FAMILY','FRIEND','MAN','WOMAN','CHILD','CHILDREN','STUDENT','TEACHER','MOTHER','FATHER','BROTHER','SISTER','HOUSE','HOME','
42 'WORLD','EARTH','WATER','FOOD','AIR','MONEY','JOB','COMPANY','OFFICE','MARKET','STORE','ROAD','CAR','BUS','TRAIN','PHONE','C
43 'NUMBER','NAME','WORD','LINE','FORM','GROUP','PART','END','POINT','LEVEL','POWER','ORDER','LAW','RULE','GOVERNMENT','SERVICE
```

Hình 4.14 Một danh sách gần 500 từ tiếng anh xuất hiện nhiều nhất trong văn bản

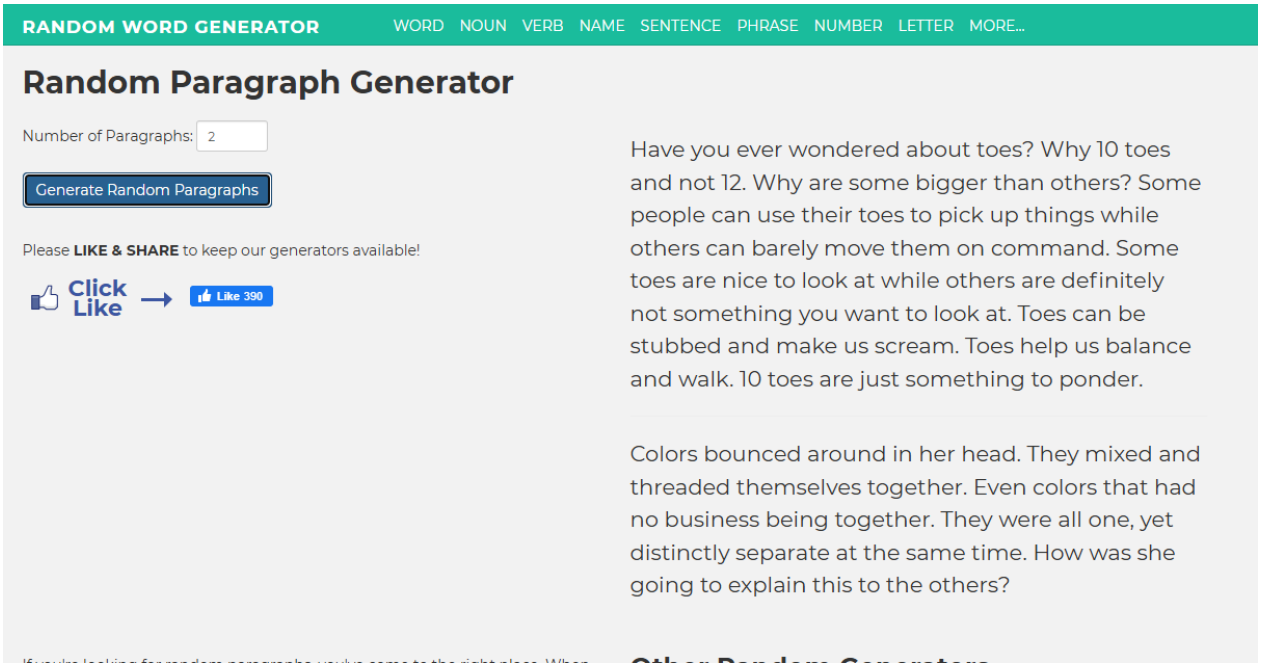
```
    chi_total = chi_squared_test(letters_only, FREQ_ENGLISH)
    cw_count = count_common_words(plaintext)
    cw_list = list_common_words_in_text(plaintext)

    score = (1 / (chi_total + 1)) * 100 + cw_count * 5

    return {
        "key_length": key_length,
        "key": key,
        "plaintext": plaintext,
        "chi_squared": chi_total,
        "common_words": cw_count,
        "common_words_list": cw_list,
        "score": score,
        "column_results": column_results,
    }
```

Hình 4.15 Biến score dùng để tính toán điểm thông qua chi_total và cw_count

4.2.3 Thực hiện giải mã với chương trình

Mặc dù chương trình cung cấp tính năng giải mã và mã hóa với khóa, tuy nhiên mục tiêu của đồ án là phá mã Vigenère bằng phương pháp phân tích tần suất (không biết khóa).

Tiến hành chạy chương trình ở chế dộ phân tích tần suất với bản mã được mã hóa từ văn bản ngẫu nhiên trên trang https://randomwordgenerator.com/paragraph.php



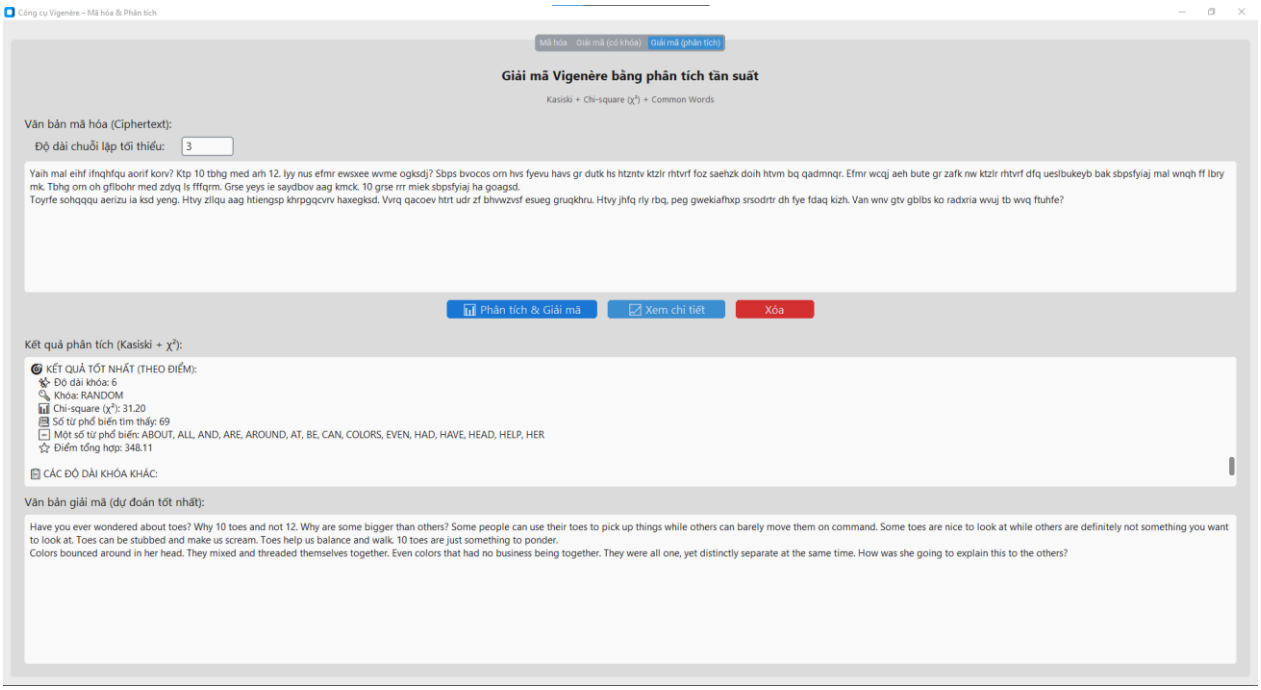
Hình 4.16 Văn bản ngẫu nhiên

Khi đó, bản mã của văn bản ngẫu nhiên trên sẽ là

"Yaih mal eihf ifnqhfqu aorif korv? Ktp 10 tbhg med arh 12. Iyy nus efmr ewsxee wvme ogksdj? Sbps bvocos orn hvs fyevu havs gr dutk hs htzntv ktzlr rhtvrf foz saehzk doih htvm bq qadmnqr. Efmr wcqj aeh bute gr zafk nw ktzlr rhtvrf dfq ueslbukeyb bak sbpsfyiaj mal wnqh ff lbry mk. Tbhg orn oh gflbohr med zdyq ls fffqrm. Grse yeys ie saydbov aag kmck. 10 grse rrr miek sbpsfyiaj ha goagsd.

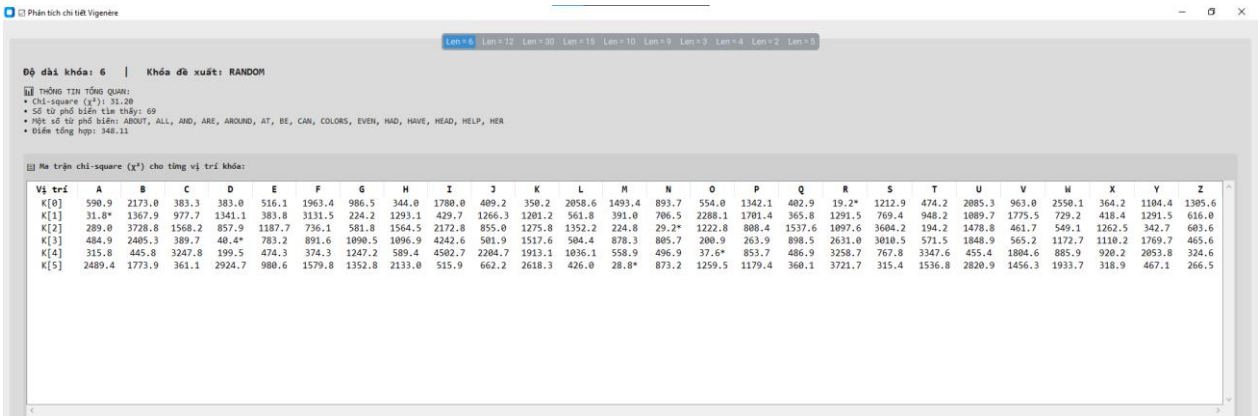
Toyrfe sohqqqu aerizu ia ksd yeng. Htvy zllqu aag htiengsp khrpgqcvrv haxegksd. Vvrq qacoev htrt udr zf bhvwzvsf esueg gruqkhru. Htvy jhfq rly rbq, peg gwekiafhxp srsodrtr dh fye fdaq kizh. Van wnv gtv gblbs ko radxria wvuj tb wvq ftuhfe?"

Sau khi chạy chương trình với độ dài chuỗi lặp lại tối thiểu là 3, ta nhận được kết quả sau:



Hình 4.17 Kết quả sau khi chạy phân tích tần suất

Dưới đây là chi tiết kết quả phân tích



Vị trí	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
K[0]	590.9	2173.0	383.3	383.0	516.1	1963.4	986.5	344.0	1780.0	409.2	350.2	2058.6	1493.4	893.7	554.0	1342.1	402.9	19.2*	1212.9	474.2	2085.3	963.0	2550.1	364.2	1104.4	1305.6
K[1]	31.8*	1367.9	977.7	1341.1	383.8	3131.5	224.2	1293.1	429.7	1266.3	1201.2	561.8	391.0	706.5	2288.1	1701.4	365.8	1291.5	769.4	948.2	1089.7	1775.5	729.2	418.4	1291.5	616.0
K[2]	289.0	3728.8	1568.2	857.9	1187.7	736.1	581.8	1564.5	2172.8	855.0	1275.8	1352.2	224.8	29.2*	1222.8	808.4	1537.6	1097.6	3604.2	194.2	1478.8	461.7	549.1	1262.5	342.7	603.6
K[3]	484.9	2405.3	389.7	40.4*	783.2	891.6	1090.5	1096.9	4242.6	501.9	1517.6	504.4	878.3	805.7	200.9	263.9	898.5	2631.0	3010.5	571.5	1848.9	565.2	1172.7	1110.2	1769.7	465.6
K[4]	315.8	445.8	3247.8	199.5	474.3	374.3	1247.2	589.4	4502.7	2204.7	1913.1	1036.1	558.9	496.9	37.6*	853.7	486.9	3258.7	767.8	3347.6	455.4	1804.6	885.9	920.2	2053.8	324.6
K[5]	2489.4	1773.9	361.1	2924.7	980.6	1579.8	1352.8	2133.0	515.9	662.2	2618.3	426.0	28.8*	873.2	1259.5	1179.4	360.1	3721.7	315.4	1536.8	2820.9	1456.3	1933.7	318.9	467.1	266.5

Hình 4.18 Chi tiết kết quả phân tích tần suất

Từ kết quả phân tích ở trên, chương trình đã xử lý toàn bộ bản mã bằng phương pháp Kasiski kết hợp kiểm định x^2 và đưa ra nhiều độ dài khóa khả thi. Dựa trên điểm số tổng hợp (bao gồm x^2 và số lượng từ tiếng Anh phổ biến), độ dài khóa phù hợp nhất được xác định là 6 ký tự với khóa thu được là "RANDOM".

Khóa này cho giá trị x^2 thấp (31.20), đồng thời bản rõ giải mã chứa số lượng lớn từ vựng hợp lệ (69 từ phổ biến) cho thấy mức độ khớp rất cao với tiếng Anh. Điều này khẳng định khóa độ dài 6 là lựa chọn chính xác nhất so với các độ dài khác như 9, 10, 12, 15 hay 30, vốn cho kết quả x^2 cao hơn và ít từ hợp lệ hơn.

Như vậy, chương trình đã tìm ra khóa "RANDOM" và giải mã được văn bản một cách hiệu quả, chứng minh tính chính xác của module phân tích và cơ chế chấm điểm tổng hợp của hệ thống.

4.3 Nhận xét và tổng kết

4.3.1 Ưu điểm của chương trình

Việc kết hợp hai phương pháp Kasiski và phân tích tần suất dựa trên kiểm định Chi bình phương đã cho thấy hiệu quả rõ rệt trong quá trình giải mã Vigenère. Kết quả thực nghiệm cho thấy chương trình không chỉ rút ngắn được thời gian ước lượng độ dài khóa mà còn xác định được khóa chính xác hơn nhờ sự hỗ trợ của các phép đo thống kê. Điều này thể hiện rõ trong bài phân tích: mặc dù có nhiều độ dài khóa khả thi, hệ thống đã lựa chọn được độ dài khóa 6 với khóa "RANDOM" – phương án sở hữu giá trị x^2 thấp nhất và chứa nhiều từ tiếng Anh hợp lệ nhất trong plaintext.

Bên cạnh đó, việc bổ sung kiểm định Chi bình phương trên plaintext sau giải mã giúp loại bỏ các khóa sai bằng cách đo mức độ khớp giữa văn bản giải mã và phân bố tần suất chuẩn của tiếng Anh. Đồng thời, chương trình còn tích hợp danh sách 500 từ tiếng Anh xuất hiện phổ biến nhất để đếm các từ hợp lệ trong plaintext từ đó tăng thêm độ tin cậy khi so sánh giữa các khóa. Hai chỉ số này được tổng hợp thành một score và score cao nhất chính là yếu tố giúp chương trình xác định khóa đúng một cách khách quan và tự động.

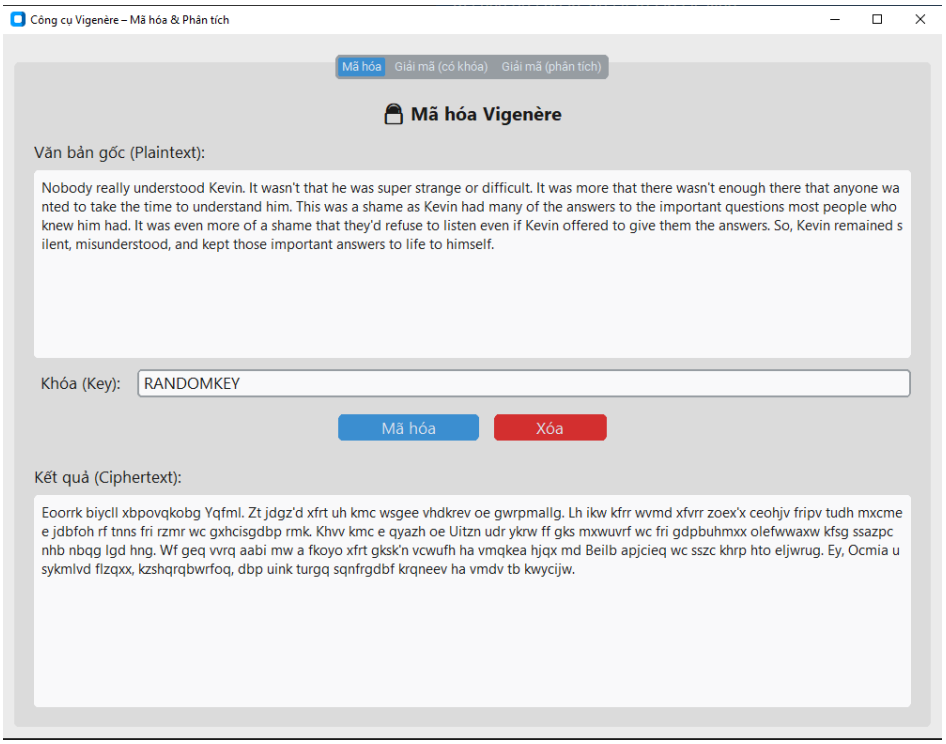
Nhờ các cơ chế này, chương trình có thể xử lý các thao tác phức tạp một cách hoàn toàn tự động, giảm đáng kể sai số do nhiễu và tăng độ chính xác trong việc giải mã. Cách tiếp cận này hoàn toàn có khả năng mở rộng để áp dụng cho các dòng mật mã cổ điển tương tự, hỗ trợ hiệu quả cho học tập và nghiên cứu trong lĩnh vực mật mã học.

4.3.2 Một số hạn chế của chương trình

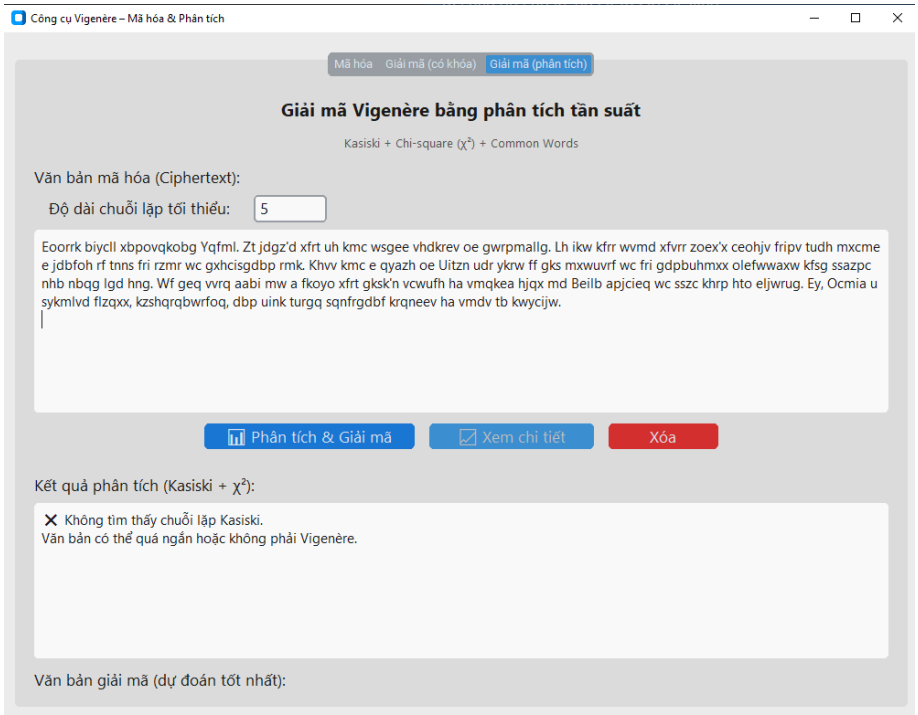
Mặc dù chương trình đạt được kết quả khả quan, nhưng vẫn tồn tại một số hạn chế nhất định. Thứ nhất, khi ciphertext xuất hiện quá nhiều chuỗi lặp có độ dài lớn, việc phân tích Kasiski có thể trở nên nhiễu và dẫn tới suy đoán sai về độ dài khóa. Do đó, cần lựa chọn ngưỡng độ dài chuỗi lặp hợp lý (thường từ 5 đến 10 ký tự) để tránh tạo ra quá nhiều ước số gây rối nhiễu trong quá trình phân tích.

Thứ hai, chương trình không thể đảm bảo giải mã chính xác 100% trong mọi trường hợp. Những bản mã quá ngắn, ít lặp hoặc có phân bố ký tự không điển hình sẽ khiến cả Kasiski lẫn kiểm định x^2 khó đưa ra kết luận chính xác. Điều này đặc biệt đúng với các ciphertext chứa ít từ tiếng Anh phổ biến, khiến cơ chế kiểm tra từ vựng mất hiệu quả.

Một ví dụ đáng chú ý mà nhóm đã gặp phải trong quá trình thử nghiệm là trường hợp khóa “RANDOMKEY”. Mặc dù khóa này ngắn hơn bản rõ rất nhiều nhưng chương trình lại không thể giải mã chính xác. Theo lý thuyết, một khóa ngắn như vậy hoàn toàn có thể bị phân tích bằng Kasiski và phân tích tần suất. Tuy nhiên ở trường hợp cụ thể này, cấu trúc phân bố ký tự của bản mã không tạo ra đủ chuỗi lặp ổn định khiến quá trình ước lượng khóa trở nên sai lệch. Đây là minh chứng rõ ràng cho việc không phải mọi bản mã đều phù hợp với các phương pháp phá mã cổ điển, đặc biệt khi dữ liệu không đáp ứng các điều kiện thống kê cần thiết.



Hình 4.19 Mã hóa bản rõ với khóa RANDOMKEY



Hình 4.20 Trường hợp điển hình khiến cho chương trình không giải mã được

Những hạn chế này xuất phát từ đặc tính của chính phương pháp Vigenère và các kỹ thuật phân tích cổ điển, chứ không chỉ từ bản thân chương trình. Tuy vậy, trong đa số trường hợp có độ dài ciphertext đủ lớn và cấu trúc phù hợp, chương trình vẫn cho kết quả rất chính xác.

4.3.3 Nhận xét chung

Qua quá trình xây dựng và thử nghiệm, có thể nhận thấy rằng chương trình giải mã Vigenère đã hoạt động hiệu quả trong phần lớn các trường hợp, đặc biệt khi bản mã có độ dài đủ lớn và phân bố ký tự phù hợp. Việc kết hợp Kasiski, phân tích tần suất và kiểm định Chi bình phương đã tạo ra một quy trình phân tích tương đối toàn diện, vừa dựa trên đặc trưng cấu trúc của bản mã, vừa dựa trên các tiêu chí thống kê mang tính khách quan. Nhờ đó, chương trình có khả năng tự động suy đoán khóa một cách chính xác hơn so với việc áp dụng đơn lẻ từng phương pháp.

Tuy nhiên, các thử nghiệm cũng cho thấy chương trình vẫn chịu ảnh hưởng bởi những hạn chế vốn có của phương pháp phá mã cổ điển. Một số bản mã đặc biệt cho thấy rằng khi chuỗi lặp không đủ ổn định hoặc bản rõ không tuân theo phân bố ký tự tiếng Anh thông thường, việc ước lượng khóa có thể gặp sai lệch. Điều này phản ánh bản chất thống kê của thuật toán: hiệu quả cao trong điều kiện lý tưởng nhưng không đảm bảo tuyệt đối trong mọi hoàn cảnh.

Nhìn chung, chương trình đã đạt được mục tiêu đặt ra: tự động hóa quy trình phân tích, hỗ trợ học tập và mang lại kết quả giải mã chính xác trong đa số trường hợp. Đồng thời, những hạn chế được phát hiện trong quá trình thử nghiệm cũng gợi mở các hướng cải tiến tiềm năng như tăng cường mô hình ngôn ngữ, sử dụng từ điển lớn hơn hoặc kết hợp nhiều thuật toán phân tích nâng cao hơn để tăng độ tin cậy.

TÀI LIỆU THAM KHẢO

[1] Simon, S. (2000). The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography. Fourth Estate.

[2] Stinson, D. R. (2021). Cryptography: Theory and Practice (4th ed.). CRC Press.

[3] Bellare, M., & Rogaway, P. (2005). Introduction to Modern Cryptography.

[4] Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). Numerical Recipes: The Art of Scientific Computing (3rd ed.). Cambridge University Press.

[5] Crypto Corner. (n.d.). The Chi-Squared Test in Cryptanalysis. Truy cập từ https://www.cryptocipher.com/chi-squared-test-in-cryptanalysis

[6] GeeksforGeeks. (2023). Vigenere Cipher Decryption using Kasiski Examination. Truy cập từ https://www.geeksfor-geeks.org/vigenere-cipher-decryption-using-kasiski-examination/

[7] Wikipedia. (n.d.). Frequency analysis. Truy cập từ https://en.wikipedia.org/wiki/Frequency_analysis

[8] Nguồn tham khảo từ ChatGPT 5.1, Claude sonnet 4.5.

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	MSSV	Nội dung phụ trách
Đào Phan Quốc Anh	2033230011	Chương 1, (4.3)
Phạm Văn Quý	2033230242	Chương 2
Võ Phát Đạt	2033230063	Chương 3, Chương 4 (4.1, 4.2), Code